

Raport Comparativ: Racket vs. Scala

Radoi Vlad, Mincu Alin, Noje Vlad, Jugaru Paul

Acest scurt raport analizează paradigmele utilizate de Racket (limbaj dinamic, bazat pe macro-uri) și Scala (limbaj hibrid FP/OOP, static tipizat) în implementarea aceluiași calculator.

A. Sintaxa pentru definirea tipurilor de date

- **Scala (Algebraic Data Types - ADTs):** Scala excelează prin cuvântul cheie `sealed trait` combinat cu case `class`. Această abordare împiedică extinderea accidentală a tipului de date în afara fișierului și asigură exhaustivitatea la compilare. Tipurile sunt clare, puternic tipizate, iar structura arborescentă este inerentă.
- **Racket (Structuri):** Racket definește tipurile prin `(struct nume (campuri))`. Fiind un limbaj cu tipare dinamică, structurile nu impun constrângeri stricte de tip asupra parametrilor `e1` sau `e2` (ar putea primi un string accidental, iar eroarea s-ar vedea abia la rulare).

B. Pattern Matching

- **Scala:** Destructurarea case `class`-urilor cu `match` este nativă, elegantă și sigură. Dacă se adaugă un nou operator în structură, compilatorul va avertiza imediat asupra locurilor unde ramura de `match` nu acoperă noua structură (exhaustiveness checking).
- **Racket:** Oferă biblioteca `racket/match`. Sintaxa este foarte curată și permite destructurarea listelor complexe foarte concis `[(list '+ e1 e2) ...]`. Racket poate scrie parsere scurte prin potrivirea directă a listelor, compensând lipsa tipajului static.

C. Tratarea erorilor

- **Scala (Monada Either):** Implementarea evită complet excepțiile try-catch în timpul evaluării. Folosind `Either[String, Double]` și maparea for-comprehension, erorile (ex: împărțirea la zero) scurtcircuitează evaluarea ramurilor de execuție, trecând elegant eroarea până la afișare (Purely Functional Error Handling).
- **Racket (with-handlers / Condiții):** Racket utilizează propagarea dinamică a excepțiilor prin `(error ...)`, pe care apoi le prindem în REPL cu `(with-handlers ...)`. Este o abordare mai orientată spre control-flow imperativ/LISP clasic. Codul este mai curat la citire (lipsește monadele), dar poate ascunde bug-uri de runtime.

D. Gestionarea Recursivității

- Ambele limbaje folosesc recursivitatea pe structura arborescentă pentru a evalua și parsa expresiile.
- Niciuna dintre funcțiile de evaluare `eval`-`expresie` scrise mai sus nu este optimizabilă complet prin Tail-Call (TCO) în niciunul dintre limbaje, deoarece apelul recursiv nu este ultima operație efectuată (rezultatele trebuie adunate sau înmulțite). Cu toate acestea, arborele matematic comun este mult prea superficial pentru a declanșa StackOverflow pe JVM (Scala) sau în Racket.

E. Care limbaj a oferit un feedback mai clar la erori?

- **În timpul dezvoltării (Compile-Time):** **Scala** este câștigător detașat. Datorită tipajului static puternic, Scala a identificat greșelile de tip, câmpurile lipsă sau "match not exhaustive" încă din IDE, permițând scrierea codului cu siguranță absolută.
- **La executare (Run-Time):** Ambele oferă mesaje bune prin designul codului. Însă tipul monadei `Either` în **Scala** forțează programatorul să trateze cazul de eroare în mod explicit (în REPL trebuie verificat `Left` vs `Right`). În Racket, e foarte ușor să uiți să pui `with-handlers`, ducând la "prăbușirea" programului în momentul erorii. Astfel, Scala oferă mecanisme structurale superioare de prevenție a bug-urilor.